

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Компьютерные системы и сети (КСиС)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

**ПРОГРАММНОЕ СРЕДСТВО
«TASK MANAGER»**

БГУИР КП 6-05-0612-01 022 ПЗ

Студент

Ушаков А.Д.

Руководитель

Шамына А.Ю.

Минск 2025

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Обзор аналогов	6
1.2 Постановка задачи	8
2 Проектирование программного средства	10
2.1 Структура программы	10
2.2 Проектирование интерфейса программного средства	11
2.3 Проектирование функционала программного средства	13
3 Разработка программного средства	17
3.1 Запуск и инициализация приложения	17
4 Тестирование программного средства	22
5 Руководство пользователя	25
5.1 Интерфейс программного средства	25
5.2 Управление программным средством	27
Заключение	30
Список использованной литературы	31
Приложение А. Текст программы	32

ВВЕДЕНИЕ

В современном мире эффективное управление задачами является ключевым фактором успеха в любой организации или проекте. Разработка системы управления задачами (Task Manager) с использованием современных технологий представляет собой актуальную задачу, которая позволяет оптимизировать рабочие процессы и повысить продуктивность.

Данный проект представляет собой веб-приложение для управления задачами, разработанное с использованием архитектуры GraphQL. GraphQL был выбран как современный язык запросов для API, который предоставляет более гибкий и эффективный способ взаимодействия между клиентом и сервером по сравнению с традиционным REST API.

Проект имеет клиент-серверную архитектуру, где серверная часть реализована с использованием Node.js и GraphQL, а клиентская часть представляет собой современное веб-приложение. Основными преимуществами выбранного подхода являются единая точка входа для всех запросов данных, возможность получения только необходимых данных в одном запросе, строгая типизация данных, автоматическая генерация документации API и улучшенная производительность за счет уменьшения количества запросов.

Целью данного проекта является создание эффективного инструмента для управления задачами, который позволит пользователям создавать и отслеживать задачи, назначать исполнителей, устанавливать приоритеты и сроки, отслеживать прогресс выполнения, а также организовывать задачи по проектам и категориям.

В пояснительной записке будут рассмотрены архитектура системы, технологический стек, реализация основных функций, особенности работы с GraphQL, процесс разработки и тестирования, а также инструкции по развертыванию. Данный проект демонстрирует практическое применение современных веб-технологий и представляет собой полноценное решение для управления задачами, которое может быть использовано как в небольших командах, так и в крупных организациях.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор аналогов

Существует множество разновидностей таск-менеджеров. Они представлены в виде десктопных и веб-приложений. Каждый имеет свои преимущества и недостатки.

1.1.1 Программное средство Trello

Trello – это визуальная система управления задачами, основанная на методологии канбан. Система представляет собой набор досок, на которых размещаются карточки задач, организованные в колонки. Каждая карточка может содержать подробное описание задачи, чек-листы, вложения файлов, комментарии, метки и информацию о сроках. Trello предлагает гибкую систему прав доступа, позволяющую настраивать видимость досок и карточек для разных пользователей. Система поддерживает интеграцию с популярными сервисами, такими как Google Drive, Slack, GitHub и другими. Trello особенно эффективен для небольших команд и проектов, где важна визуальная организация работы. Система предлагает бесплатный базовый план с ограниченным набором функций и платные версии (Standard, Premium, Enterprise) с расширенными возможностями, включая неограниченное количество досок, расширенные права доступа, автоматизацию и аналитику.

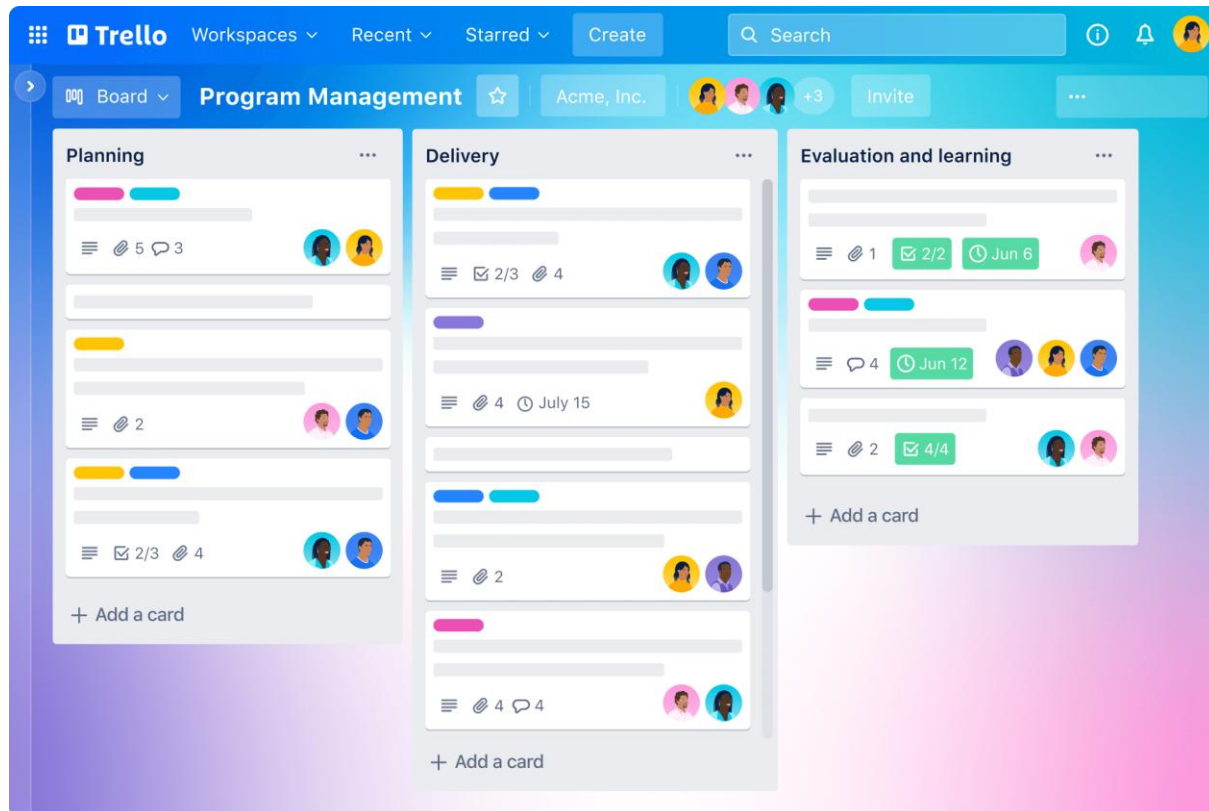


Рисунок 1.1 – Программное средство Trello

1.1.2 Веб-приложение Asana

Asana – это комплексная платформа для управления проектами и задачами, которая предлагает множество способов организации и отслеживания работы. Система поддерживает различные представления данных: список задач, канбан-доски, календарь и временная шкала (Gantt-диаграмма). Asana позволяет создавать иерархическую структуру проектов, задач и подзадач, назначать исполнителей, устанавливать сроки и приоритеты. Система предлагает мощные инструменты автоматизации, позволяющие создавать триггеры и действия для автоматизации рутинных задач. Asana включает в себя функции отслеживания времени, создания форм для сбора информации, управления портфелем проектов и генерации отчетов. Платформа поддерживает интеграцию с более чем 200 сервисами, включая Slack, Microsoft Teams, Google Workspace и другие. Asana предлагает несколько тарифных планов (Basic, Premium, Business, Enterprise), каждый из которых предоставляет различные возможности для команд разного размера и потребностей.

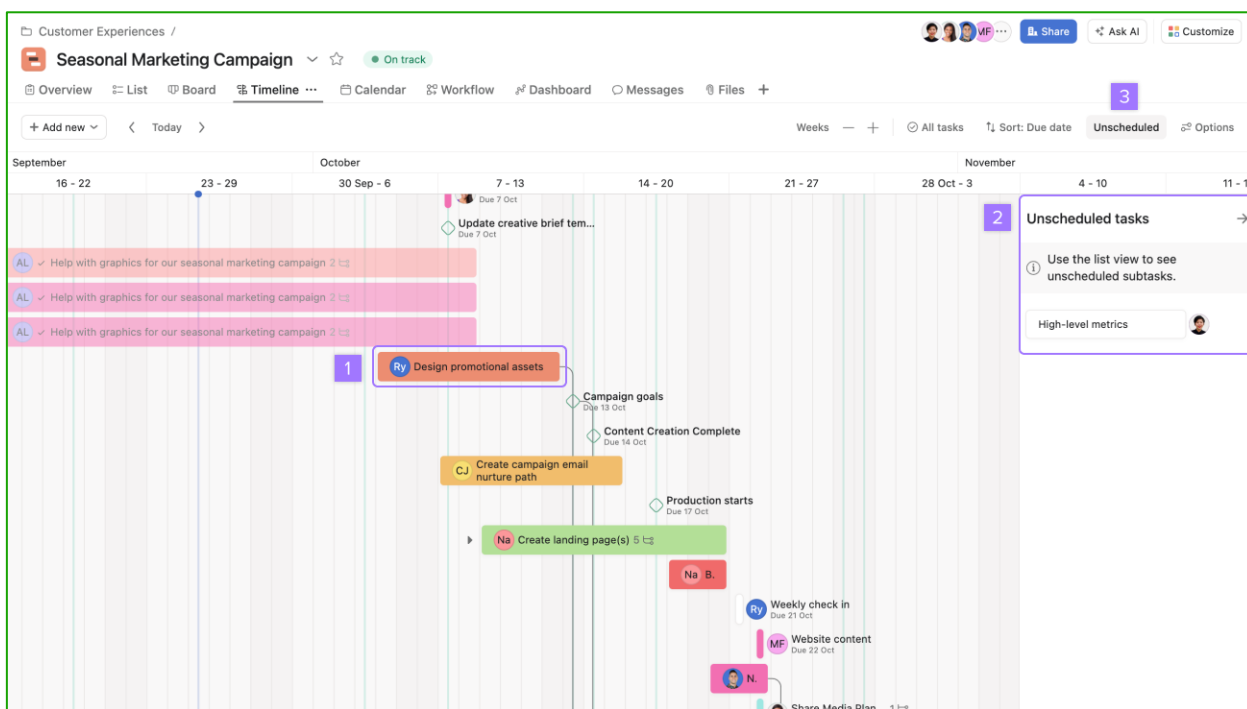


Рисунок 1.2 – Веб-приложение Asana

1.1.3 Веб-приложение Jira

Jira – это профессиональный инструмент для управления проектами, разработанный компанией Atlassian и особенно популярный среди команд разработчиков. Система предлагает гибкую настройку рабочих процессов, позволяющую адаптировать её под различные методологии разработки, включая Scrum, Kanban и гибридные подходы. Jira позволяет создавать

различные типы задач (баги, истории пользователей, эпика, задачи), настраивать их поля и статусы, устанавливать связи между задачами и отслеживать зависимости. Система включает в себя мощные инструменты для планирования спринтов, управления бэклогом, проведения ежедневных встреч и ретроспектив. Jira предлагает расширенные возможности для отслеживания времени, создания пользовательских отчетов и дашбордов, а также интеграцию с системами контроля версий (Git, SVN) и CI/CD-инструментами. Платформа поддерживает автоматизацию рабочих процессов через Jira Automation и интеграцию с другими продуктами Atlassian (Confluence, Bitbucket) и сторонними сервисами. Jira доступна в нескольких версиях: Jira Software (для команд разработчиков), Jira Service Management (для IT-служб) и Jira Work Management (для бизнес-команд), каждая из которых может быть развернута в облаке или локально.

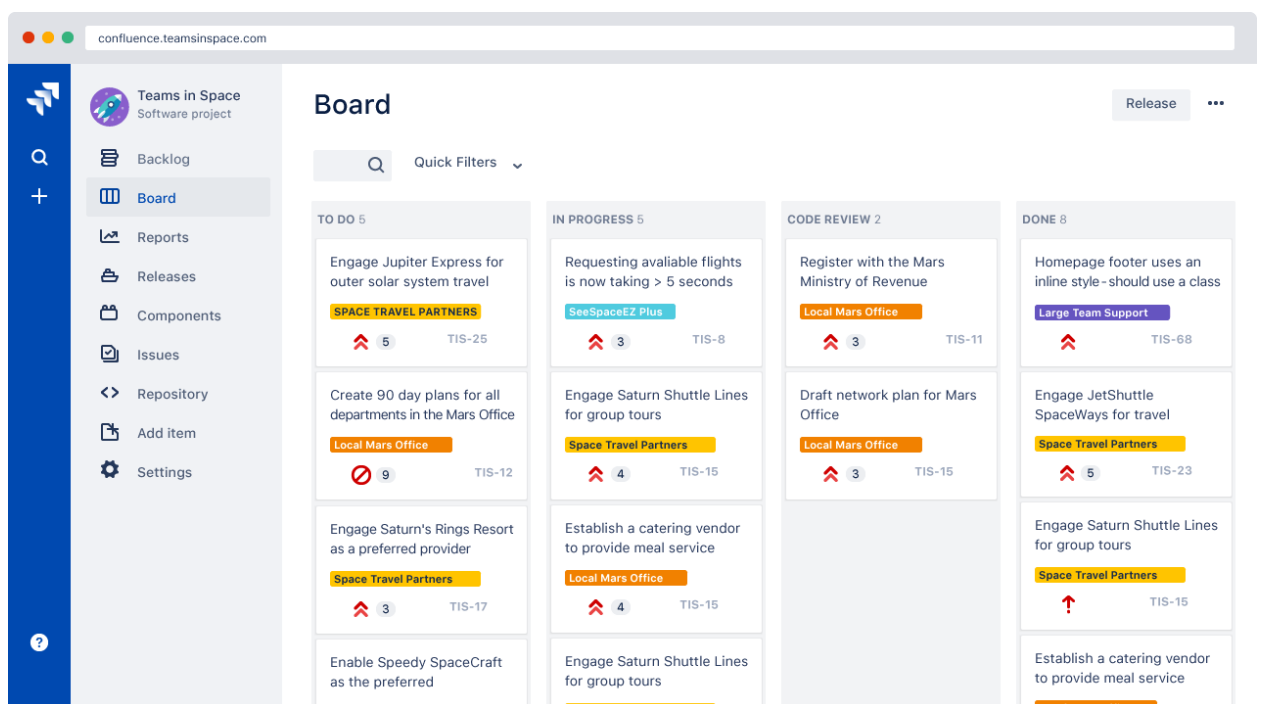


Рисунок 1.3 – Веб-приложение Jira

1.2 Постановка задачи

На основе проведенного анализа проекта принято решение разработать веб-приложение Task Manager, обладающее следующими функциями:

- создание, редактирование и удаление задач с указанием заголовка и описания;
- установка статуса выполнения задачи (выполнена/не выполнена);
- назначение сложности задачи (легкая, средняя, сложная);
- добавление тегов к задачам для категоризации;
- установка сроков выполнения задач;

- фильтрация задач по сложности;
- фильтрация задач по тегам;
- фильтрация задач по диапазону дат;
- регистрация и авторизация пользователей;
- привязка задач к конкретным пользователям;
- отслеживание даты создания задач.

Приложение будет доступно пользователям на русском языке. Для разработки веб-приложения будет использоваться:

- серверная часть: Node.js с Apollo Server Express и GraphQL;
- клиентская часть: React.js;
- среда разработки: Visual Studio Code.

Приложение будет иметь современный пользовательский интерфейс, обеспечивающий удобное взаимодействие с системой, и будет оптимизировано для работы как на десктопных, так и на мобильных устройствах.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Структура программы

При разработке веб-приложения Task Manager используется клиент-серверная архитектура. Сервер реализован на Node.js с использованием Apollo Server Express и GraphQL. Клиентская часть разработана с использованием React.js.

2.1.1 При разработке сервера используются следующие модули:

- схема GraphQL (typeDefs.js):
 - а) типы данных:
 - User – описывает пользователя системы;
 - Task – описывает задачу;
 - AuthPayload – описывает данные авторизации;
 - б) перечисления:
 - TaskComplexity – определяет уровни сложности задач (EASY, MEDIUM, HARD);
 - в) запросы (Query):
 - tasks – получение списка всех задач;
 - task – получение задачи по ID;
 - me – получение данных текущего пользователя;
 - tasksByComplexity – фильтрация задач по сложности;
 - tasksByTag – фильтрация задач по тегу;
 - tasksByDateRange – фильтрация задач по диапазону дат;
 - г) мутации (Mutation):
 - createTask – создание новой задачи;
 - updateTask – обновление существующей задачи;
 - deleteTask – удаление задачи;
 - register – регистрация нового пользователя;
 - login – авторизация пользователя;
- резолверы (resolvers):
 - а) Query resolvers – обработчики запросов для получения данных;
 - б) Mutation resolvers – обработчики мутаций для изменения данных;

2.1.2 При разработке клиента используются следующие модули:

- компоненты:
 - а) Auth.js – компонент авторизации и регистрации пользователей;
 - б) TaskList.js – компонент отображения списка задач с возможностью фильтрации;
 - в) CreateTask.js – компонент создания и редактирования задач;
 - г) AddTask.js – компонент быстрого добавления задачи;
- контекст (context):
 - а) Apollo Client – для работы с GraphQL API;

b) Контекст аутентификации – для управления состоянием авторизации;

– GraphQL:

a) Запросы и мутации – для взаимодействия с сервером;

b) Типы данных – для типизации данных на клиенте;

– стили:

a) App.css – основные стили приложения;

b) index.css – глобальные стили;

2.1.3 Серверная часть обеспечивает:

– обработку GraphQL запросов и мутаций;

– валидацию данных;

– аутентификацию пользователей;

– бизнес-логику работы с задачами;

2.1.4 Клиентская часть обеспечивает:

– пользовательский интерфейс;

– взаимодействие с сервером через GraphQL;

– управление состоянием приложения;

– валидацию форм;

– фильтрацию и отображение задач.

2.2 Проектирование интерфейса программного средства

При разработке программного средства за основу будет взят дизайн, обеспечивающий понятное и интуитивное взаимодействие с пользователями.

2.2.1 Главное окно

Главное окно приложения состоит из трех основных частей. В верхней части окна располагается шапка (header), содержащая название приложения "Task Manager" и кнопку выхода из системы (logout). Шапка имеет белый фон, скругленные углы и тень для визуального выделения.

Под шапкой размещается форма создания новой задачи (CreateTask), которая представляет собой карточку с белым фоном, тенью и скругленными углами. Форма содержит следующие элементы:

- Поле ввода заголовка задачи

- Текстовое поле для описания задачи

- Две строки с полями ввода, каждая из которых содержит по два элемента:

- Первая строка: выбор даты и уровня сложности (Easy, Medium, Hard)

- Вторая строка: ввод тегов (через запятую) и установка срока выполнения

- Кнопка "Add Task" для создания новой задачи

В основной части окна располагается список задач (TaskList). Каждая задача отображается в виде отдельной карточки, которая содержит:

- Чекбокс для отметки о выполнении задачи
- Заголовок задачи
- Описание задачи
- Метаданные задачи, включающие:
 - Дату создания (с иконкой календаря)
 - Уровень сложности (с иконкой молнии)
 - Теги (отображаются как синие метки с хештегом)
 - Срок выполнения (с иконкой часов)
 - Время создания задачи
- Кнопки управления: "Edit" и "Delete"

При нажатии на кнопку "Edit" карточка задачи трансформируется в форму редактирования, которая содержит те же поля, что и форма создания задачи, но с предзаполненными значениями текущей задачи. Форма редактирования имеет кнопки "Save" и "Cancel".

Выполненные задачи отображаются с пониженной прозрачностью (0.7) и зачеркнутым заголовком. Все интерактивные элементы (кнопки, поля ввода) имеют эффекты при наведении и нажатии для улучшения пользовательского опыта.

Приложение использует адаптивный дизайн с максимальной шириной контента 1200 пикселей и отступами по бокам. Цветовая схема построена на использовании переменных CSS:

- Основной цвет: #4a90e2 (синий)
- Фоновый цвет: #f5f6fa (светло-серый)
- Цвет текста: #2c3e50 (темно-серый)
- Цвет ошибки: #e74c3c (красный)
- Цвет успеха: #2ecc71 (зеленый)

Все элементы интерфейса имеют единообразное оформление с использованием скругленных углов (8px), теней и плавных анимаций при взаимодействии. Новая фигура в систему. Макет главного окна представлен на рисунке 2.1

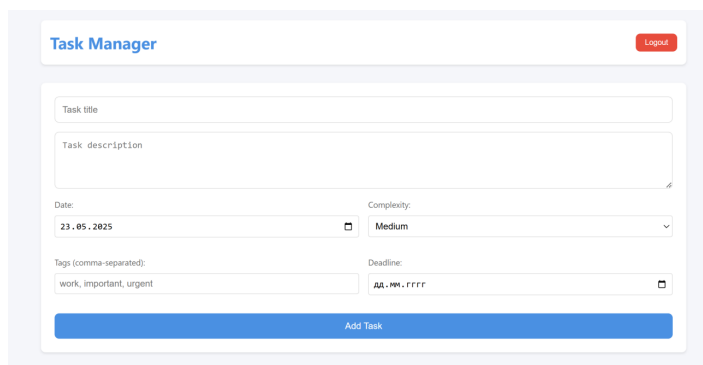
The image shows a web form titled "Task Manager" in a light blue header. In the top right corner of the header is a red "Logout" button. The form contains several input fields: "Task title" (a single-line text box), "Task description" (a multi-line text area), "Date:" (a date picker showing "23.05.2025"), "Complexity:" (a dropdown menu showing "Medium"), "Tags (comma-separated):" (a text box with "work, important, urgent"), and "Deadline:" (a date picker showing "dd.mm.yyyy"). At the bottom of the form is a prominent blue button labeled "Add Task".

Рисунок 2.1 – Главное окно приложения

2.3 Проектирование функционала программного средства

Приложение Task Manager предоставляет комплексный набор функций для эффективного управления и организации задач. В основе приложения лежит возможность создания новых задач с детальной информацией, включающей заголовок, описание и различные метаданные. Каждой задаче можно присвоить уровень сложности (Легкий, Средний или Сложный) и добавить пользовательские метки для лучшей организации и фильтрации.

Приложение реализует надежную систему управления задачами, которая позволяет пользователям изменять существующие задачи через интуитивно понятный интерфейс редактирования. Пользователи могут обновлять любой аспект задачи, включая её заголовок, описание, уровень сложности и связанные метки. Система поддерживает полную историю создания и изменений задач через временные метки, обеспечивая правильное отслеживание жизненного цикла задачи.

Ключевой особенностью приложения являются его sophisticated возможности фильтрации и организации задач. Пользователи могут фильтровать задачи по нескольким критериям, включая уровень сложности, конкретные метки и временные диапазоны. Это позволяет эффективно организовывать и расставлять приоритеты задач на основе различных параметров. Приложение также поддерживает отслеживание выполнения задач через систему чекбоксов, позволяя пользователям отмечать задачи как выполненные, сохраняя при этом их историю в системе.

Приложение реализует безопасную систему аутентификации, которая гарантирует, что каждый пользователь может получить доступ и изменять только свои задачи. Это достигается с помощью механизма аутентификации на основе токенов, который поддерживает пользовательские сессии и защищает данные задач. Система также предоставляет функциональность регистрации и входа пользователей, создавая персонализированный опыт для каждого пользователя.

Управление задачами улучшено за счет реализации сроков выполнения, позволяя пользователям устанавливать конкретные даты выполнения для своих задач. Приложение отображает эти сроки заметно в интерфейсе задачи, помогая пользователям эффективно расставлять приоритеты в работе. Кроме того, система поддерживает массовые операции с задачами, позволяя пользователям выполнять действия с несколькими задачами одновременно. Управление данными в приложении осуществляется через GraphQL API, который обеспечивает эффективные и гибкие возможности запроса данных. Это позволяет обновлять информацию о задачах в реальном времени и гарантирует, что все изменения немедленно отражаются в пользовательском интерфейсе. Система поддерживает согласованность данных через надлежащие механизмы валидации и обработки ошибок.

Пользовательский интерфейс разработан интуитивно понятным и отзывчивым, обеспечивая мгновенную обратную связь для всех действий

пользователя. Задачи отображаются в четком, организованном виде с визуальными индикаторами для различных состояний (выполнено, в ожидании) и уровней сложности. Интерфейс включает возможности поиска и фильтрации, что позволяет пользователям легко находить конкретные задачи или группы задач на основе различных критериев.

Приложение реализует надлежащую обработку ошибок и валидацию во всей системе, обеспечивая целостность данных и предоставляя пользователям осмысленную обратную связь при возникновении ошибок. Это включает валидацию входных данных задач, правильную обработку ошибок аутентификации и корректную обработку сетевых проблем. Система также поддерживает правильное управление состоянием, гарантируя, что пользовательский интерфейс всегда отражает текущее состояние задач.

2.3.1 Запуск сервера

Запуск сервера представлена на рисунке 2.2. Она отражает полную последовательность действий по началу работы с сервером.

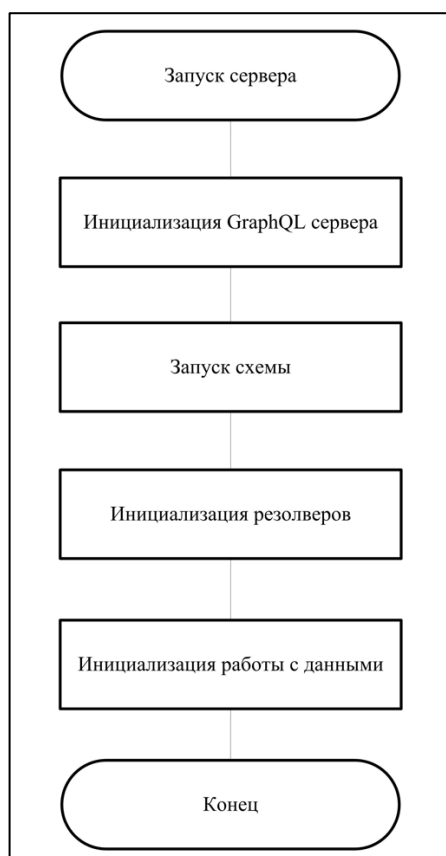


Рисунок 2.2 – Блок-схема загрузки сервера

2.3.2 Запуск клиента

Запуск клиента представлена на рисунке 2.3. Она отражает полную последовательность действий по началу работы с клиентом.



Рисунок 2.3 – Блок-схема обработчика события onmousedown инструмента фигуры

2.3.3 Схема функционирования приложения

Приложение представляет собой систему управления задачами, построенную на React и Node.js. Каждая задача содержит заголовок, описание, уровень сложности, теги и сроки выполнения. Пользователи могут создавать, редактировать и удалять задачи, а также фильтровать их по сложности, тегам и датам. Взаимодействие между клиентом и сервером осуществляется через GraphQL API, что обеспечивает эффективную работу с данными. Схема функционирования приложения представлена на рисунке 2.4.

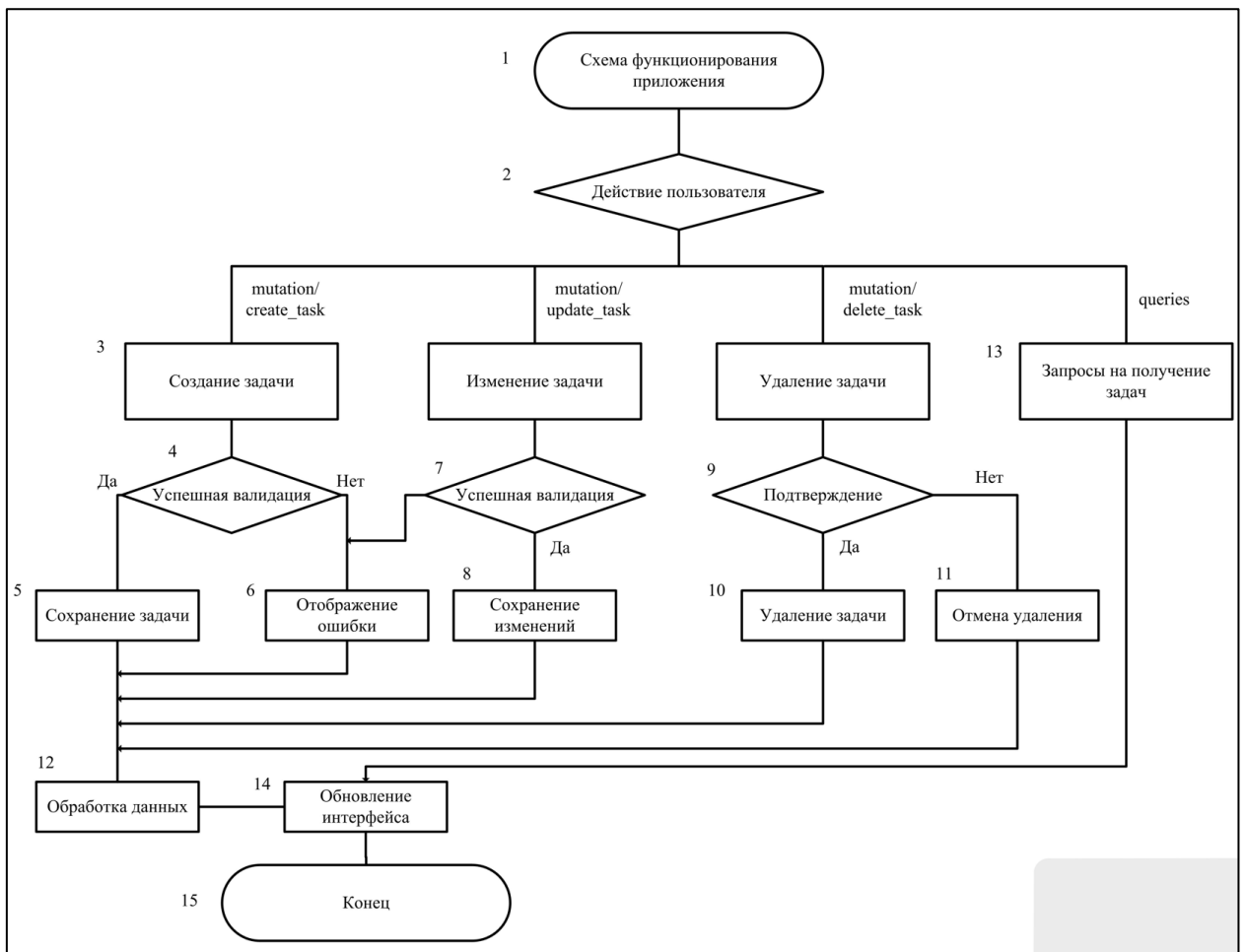


Рисунок 2.4 – Схема функционирования приложения

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Запуск и инициализация приложения

При запуске приложения происходит комплексная инициализация как серверной, так и клиентской части. Серверная часть инициализирует Express и Apollo Server, настраивает GraphQL схему и резолверы, устанавливает необходимые middleware и подготавливает систему к обработке запросов. Клиентская часть инициализирует React приложение, настраивает Apollo Client для взаимодействия с сервером, инициализирует состояние приложения и подготавливает пользовательский интерфейс к работе.

3.1.1 Инициализация сервера

Инициализация сервера происходит в файле `index.js`. Процесс начинается с создания Express приложения, которое будет служить базовым HTTP сервером для обработки всех входящих запросов. Далее происходит настройка CORS, что позволяет клиентскому приложению взаимодействовать с сервером, даже если они находятся на разных доменах. После этого создается и настраивается Apollo Server - GraphQL сервер, который будет обрабатывать все GraphQL запросы. Затем GraphQL middleware применяется к Express приложению, что позволяет интегрировать GraphQL функциональность в Express. Завершающим этапом является запуск сервера на указанном порту, после чего он начинает прослушивать входящие соединения.

Процесс инициализации сервера также включает настройку обработки ошибок, инициализацию подключения к базе данных, настройку логирования, инициализацию кэширования и настройку безопасности. Все эти компоненты работают вместе, обеспечивая стабильную и безопасную работу сервера.

Код инициализации сервера:

```
async function startServer() {  
  const app = express();  
  app.use(cors());  
  
  const server = new ApolloServer({  
    typeDefs,  
    resolvers,  
    context: ({ req }) => ({  
      req  
    })  
  })
```

```

});

await server.start();

server.applyMiddleware({ app });

const PORT = process.env.PORT || 4000;

app.listen(PORT, () => {
  console.log(`Server running at
http://localhost:${PORT}${server.graphqlPath}`);
});
}

```

3.1.2 Инициализация клиента

Инициализация клиентской части происходит в файле App.js. Процесс начинается с настройки Apollo Client - клиентской библиотеки для работы с GraphQL. Затем происходит настройка аутентификации, что подготавливает систему к работе с JWT токенами. Следующим этапом является инициализация состояния приложения, в ходе которой создается начальное состояние React приложения. Завершающим этапом является настройка маршрутизации компонентов, что подготавливает систему к отображению различных компонентов.

Процесс инициализации клиента также включает настройку темизации, инициализацию глобальных стилей, настройку обработки ошибок на клиенте, инициализацию системы уведомлений и настройку интернационализации. Все эти компоненты обеспечивают полноценную работу клиентской части приложения.

Код инициализации Apollo Client:

```

const httpLink = createHttpLink({
  uri: 'http://localhost:4000/graphql'
});

const authLink = setContext((_, { headers }) => {
  const token = localStorage.getItem('token');
  return {
    headers: {

```



```

    ...headers,

    authorization: token ? `Bearer ${token}` : ''
  }
};

});

const client = new ApolloClient({
  link: authLink.concat(httpLink),
  cache: new InMemoryCache()
});

```

3.1.3 Аутентификация и авторизация

Система аутентификации реализована с использованием JWT токенов. При успешной авторизации токен сохраняется в localStorage браузера и используется для всех последующих запросов к серверу. Система аутентификации обеспечивает регистрацию новых пользователей, авторизацию существующих пользователей, восстановление сессии, управление токенами и защиту от несанкционированного доступа.

Процесс аутентификации включает несколько этапов. Сначала происходит валидация учетных данных пользователя. После успешной валидации генерируется JWT токен, который сохраняется на клиенте. При каждом последующем запросе к серверу происходит проверка токена. Система также обеспечивает обновление токена при необходимости, что повышает безопасность приложения.

Код обработки аутентификации:

```

const handleLogin = (userData, token) => {
  localStorage.setItem('token', token);
  localStorage.setItem('userId', userData.id);
  setUser(userData);
};

const handleLogout = () => {
  localStorage.removeItem('token');

```

```
localStorage.removeItem('userId');  
  
setUser(null);  
};
```

3.1.4 Управление состоянием приложения

Управление состоянием приложения реализовано с использованием React Hooks и Apollo Client. Система управления состоянием обеспечивает работу с глобальным состоянием приложения, состоянием компонентов, кэшированием данных, синхронизацией состояния между компонентами и обработкой асинхронных операций.

В приложении поддерживаются различные состояния, включая состояние авторизации пользователя, состояние списка задач, состояние фильтрации задач, состояние создания и редактирования задачи, состояние уведомлений, состояние загрузки данных и состояние ошибок. Все эти состояния взаимодействуют между собой, обеспечивая корректную работу приложения.

3.1.5 GraphQL схема и резолверы

GraphQL схема определяет типы данных и операции, доступные в API. Схема включает определение типов данных, запросов, мутаций, подписок, входных типов и перечислений. Это позволяет создать четкую структуру API и обеспечить типобезопасность при работе с данными.

Основными элементами схемы являются типы данных User, Task и AuthPayload, запросы tasks, task, me, tasksByComplexity, tasksByTag и tasksByDateRange, а также мутации createTask, updateTask, deleteTask, register и login. Каждый из этих элементов имеет четко определенную структуру и функциональность.

3.1.6 Обработка ошибок и валидация

Система включает многоуровневую обработку ошибок, начиная с валидации данных на клиенте. На этом уровне происходит проверка обязательных полей, форматов данных, ограничений и валидация форм. Далее следует валидация GraphQL запросов, которая включает проверку структуры запросов, типов данных, прав доступа и входных параметров.

На сервере происходит обработка ошибок, включающая логирование, форматирование, обработку исключений и возврат понятных сообщений об ошибках. Пользователю ошибки отображаются в понятном формате, с подсказками по исправлению, визуальным выделением ошибок и интерактивной помощью.

3.1.7 Оптимизация производительности

Для оптимизации производительности используются различные техники. Кэширование данных на клиенте через Apollo Client обеспечивает эффективное хранение и обновление данных. Оптимизация GraphQL запросов позволяет минимизировать количество запросов и оптимизировать их структуру.

Ленивая загрузка компонентов реализована через разделение кода, динамический импорт, предзагрузку компонентов и оптимизацию бандла. Мемоизация компонентов React обеспечивает кэширование вычислений, оптимизацию ререндеринга и эффективное использование `useMemo` и `useCallback`.

3.1.8 Безопасность

Система безопасности включает комплекс мер, начиная с JWT аутентификации. Она обеспечивает безопасное хранение токенов, проверку подписи, обновление токенов и защиту от перехвата. Защита от CSRF атак реализована через проверку заголовков, валидацию источников запросов, защиту форм и использование безопасных куки.

Валидация входных данных включает санитизацию данных, проверку типов, валидацию форматов и защиту от инъекций. Безопасное хранение токенов обеспечивается через шифрование данных, защиту от XSS, использование безопасных куки и управление сессиями. Защита GraphQL эндпоинтов реализована через ограничение глубины запросов, защиту от DoS атак, валидацию запросов и контроль доступа.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

В ходе тестирования приложения Task Manager были выявлены несколько проблем и недочетов в работе веб-приложения.

Одна из основных проблем была связана с обновлением списка задач после создания новой задачи. При создании задачи через форму CreateTask, Apollo Client кэшировал данные, что приводило к тому, что новая задача не отображалась в списке до перезагрузки страницы. Проблема возникала из-за того, что Apollo Client по умолчанию использует кэширование для оптимизации производительности. Для решения этой проблемы было принято решение использовать политику fetchPolicy: 'network-only' при запросе списка задач, что заставляет Apollo Client всегда делать новый запрос к серверу. Код исправленного запроса представлен ниже:

```
const { loading, error, data } = useQuery(GET_TASKS, {  
  fetchPolicy: 'network-only'  
});
```

Другая проблема была обнаружена при работе с тегами задач. При создании или редактировании задачи, если пользователь не указывал теги, система автоматически добавляла тег 'untagged'. Однако при попытке отфильтровать задачи по тегу 'untagged' возникала ошибка, так как система не могла корректно обработать пустой массив тегов. Для решения этой проблемы была добавлена проверка на пустой массив тегов и модифицирована логика фильтрации. Код исправленной обработки тегов представлен ниже:

```
const tagsArray = tags.split(',').map(tag => tag.trim()).filter(tag =>  
tag);  
await createTask({  
  variables: {  
    // ... другие поля  
    tags: tagsArray.length > 0 ? tagsArray : ['untagged'],  
  }  
})
```

Также была выявлена проблема с обработкой дат в приложении. При создании задачи с дедлайном в прошлом, система не выдавала предупреждения пользователю. Это могло привести к созданию задач с неактуальными сроками выполнения. Для решения этой проблемы была добавлена валидация дат на клиентской стороне. Код проверки даты представлен ниже:

```
const handleSubmit = async (e) => {  
  e.preventDefault();
```

```

    if (!title.trim()) return;

    try {
      const userId = localStorage.getItem('userId');
      if (!userId) {
        console.error('User ID not found');
        return;
      }

      // Проверка даты дедлайна
      if (deadline && new Date(deadline) < new Date()) {
        console.error('Deadline cannot be in the past');
        return;
      }

      const tagsArray = tags.split(',').map(tag => tag.trim()).filter(tag
=> tag);

      await createTask({
        variables: {
          title,
          description: description || '',
          userId,
          date: date || new Date().toISOString().split('T')[0],
          complexity: complexity || 'MEDIUM',
          tags: tagsArray.length > 0 ? tagsArray : ['untagged'],
          deadline: deadline || null
        }
      });
      // ... остальной код
    } catch (err) {
      console.error('Error creating task:', err);
    }
  };
};

```

Проблема с аутентификацией пользователей была обнаружена при тестировании системы безопасности. При выходе из системы (logout) токен аутентификации удалялся из localStorage, но Apollo Client продолжал использовать кэшированные данные. Это могло привести к тому, что пользователь все еще мог видеть свои задачи даже после выхода из системы. Для решения этой проблемы была добавлена очистка кэша Apollo Client при выходе из системы. Код исправленной функции выхода представлен ниже:

```

const handleLogout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('userId');
  client.resetStore(); // Очистка кэша Apollo Client
  setUser(null);
};

```

В ходе тестирования также была выявлена проблема с обработкой ошибок при работе с GraphQL-запросами. При возникновении ошибки на сервере, клиентская часть не всегда корректно отображала сообщение об ошибке пользователю. Для улучшения обработки ошибок была добавлена более детальная обработка ошибок GraphQL. Код обработки ошибок представлен ниже:

```
const [createTask] = useMutation(CREATE_TASK, {
  refetchQueries: [{ query: GET_TASKS }],
  onError: (error) => {
    console.error('GraphQL Error:', error);
    // Отображение ошибки пользователю
    setError(error.message);
  }
});
```

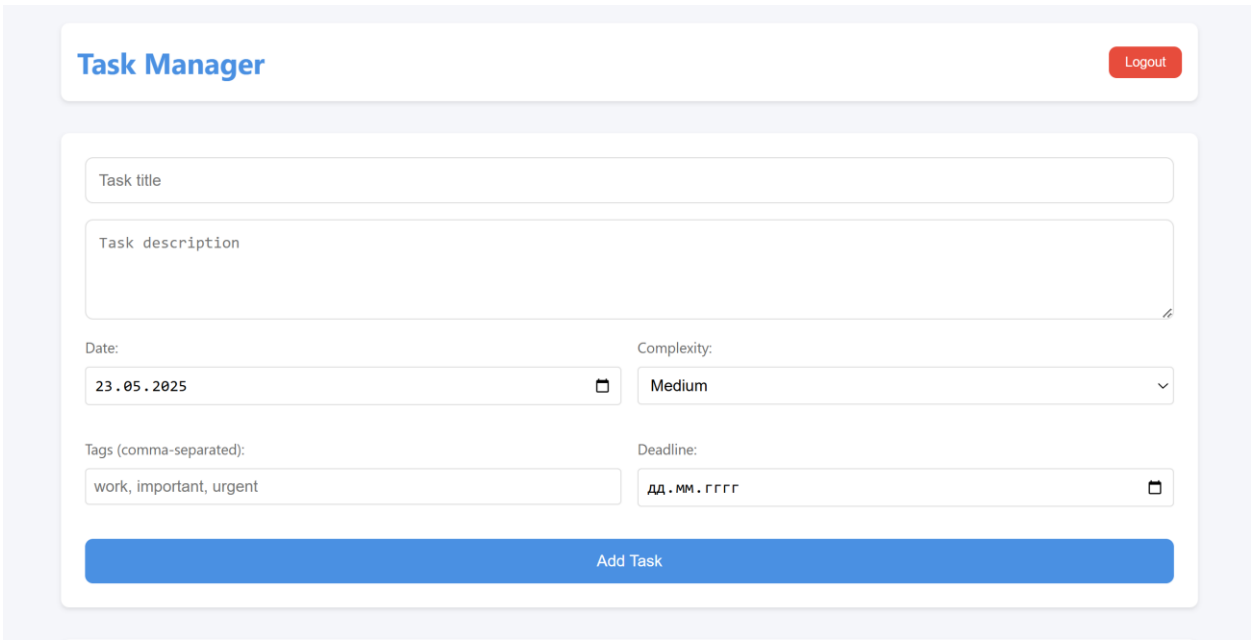
Все выявленные проблемы были успешно решены, что позволило улучшить стабильность и надежность работы приложения. Внедренные исправления также улучшили пользовательский опыт и безопасность системы.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Интерфейс программного средства

5.1.1 Главное окно

Главное окно (рис. 5.1) приложения представляет собой современный веб-интерфейс, выполненный в минималистичном стиле. В верхней части окна расположена шапка приложения (header), содержащая название приложения "Task Manager" и кнопку выхода из системы (Logout). Под шапкой находится форма создания новой задачи, которая включает в себя поля для ввода названия, описания, даты, сложности, тегов и срока выполнения задачи. Основную часть окна занимает список задач, где каждая задача представлена в виде отдельной карточки с полной информацией о ней. Интерфейс выполнен в светлой цветовой гамме с использованием теней и скругленных углов для создания современного визуального стиля.



The screenshot displays the main interface of the 'Task Manager' application. At the top, a header bar contains the application name 'Task Manager' on the left and a red 'Logout' button on the right. Below the header is a form for creating a new task. This form consists of several input fields: a text field for 'Task title', a larger text area for 'Task description', a date picker for 'Date' (currently showing 23.05.2025), a dropdown menu for 'Complexity' (currently set to Medium), a text field for 'Tags (comma-separated)' (containing 'work, important, urgent'), and a date picker for 'Deadline' (showing DD.MM.YYYY). At the bottom of the form is a prominent blue button labeled 'Add Task'.

Рисунок 5.1 – Главное окно

5.1.2 Форма создания задачи

Форма создания задачи представляет собой структурированный блок с полями ввода. В верхней части формы расположено поле для ввода названия задачи, которое является обязательным для заполнения. Под ним находится текстовое поле для описания задачи, которое может быть заполнено по желанию пользователя. Далее следуют поля для выбора даты выполнения и уровня сложности задачи (Easy, Medium, Hard). В нижней части формы расположены поля для ввода тегов (через запятую) и срока выполнения задачи. Завершает форму кнопка "Add Task" для создания новой задачи. Все поля

формы имеют четкую визуальную структуру и подписи для удобства пользователя.

5.1.3 Список задач

Список задач (рис. 5.2) представляет собой вертикальную последовательность карточек, где каждая карточка содержит информацию об отдельной задаче. Карточка задачи включает в себя чекбокс для отметки о выполнении, заголовок задачи, описание, метаданные (дата создания, сложность, теги) и кнопки управления (Edit, Delete). Выполненные задачи визуально отличаются от невыполненных: их заголовок перечеркивается, а прозрачность карточки снижается. Каждая карточка имеет тень и скругленные углы, что создает эффект "парящего" элемента на странице.

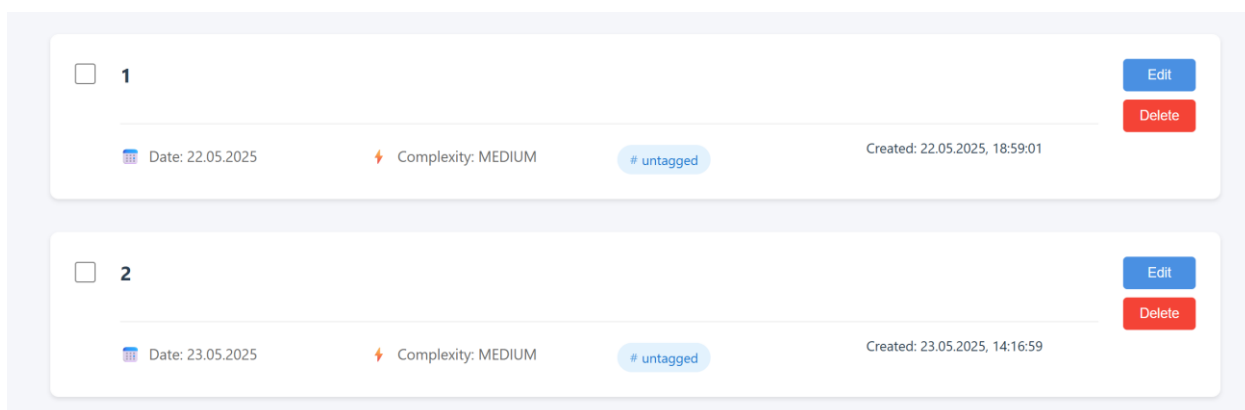


Рисунок 5.2 – Список задач

5.1.4 Форма редактирования задачи

Форма редактирования задачи (рис. 5.3) появляется при нажатии на кнопку "Edit" в карточке задачи. Она содержит те же поля, что и форма создания задачи, но с предзаполненными значениями текущей задачи. Форма редактирования отображается непосредственно в карточке задачи, заменяя её содержимое. В нижней части формы расположены кнопки "Save" для сохранения изменений и "Cancel" для отмены редактирования. Форма имеет легкий фоновый цвет для визуального выделения на странице.

The image shows a form for editing a task, overlaid on a card. The form has a light gray background. It starts with a checkbox and the number '1'. Below it is a text input field labeled 'Task description'. Underneath that are two rows of form fields. The first row has 'Date:' followed by a text input containing '22.05.2025' and a 'Complexity:' dropdown menu showing 'Medium'. The second row has 'Tags:' followed by a text input containing 'untagged' and 'Deadline:' followed by a text input containing '2025-05-20T12:00:00'. At the bottom of the form are two buttons: a green 'Save' button and a red 'Cancel' button.

Рисунок 5.3 – Форма редактирования задачи

5.1.5 Форма авторизации

Форма авторизации отображается при первом входе в систему или после выхода из неё. Она представляет собой центрированный блок с полями для ввода учетных данных пользователя. Форма содержит поля для ввода имени пользователя и пароля, а также кнопку входа в систему. В нижней части формы предусмотрена возможность переключения между режимами входа и регистрации. Форма авторизации имеет четкую визуальную структуру и информативные сообщения об ошибках при неудачной попытке входа.

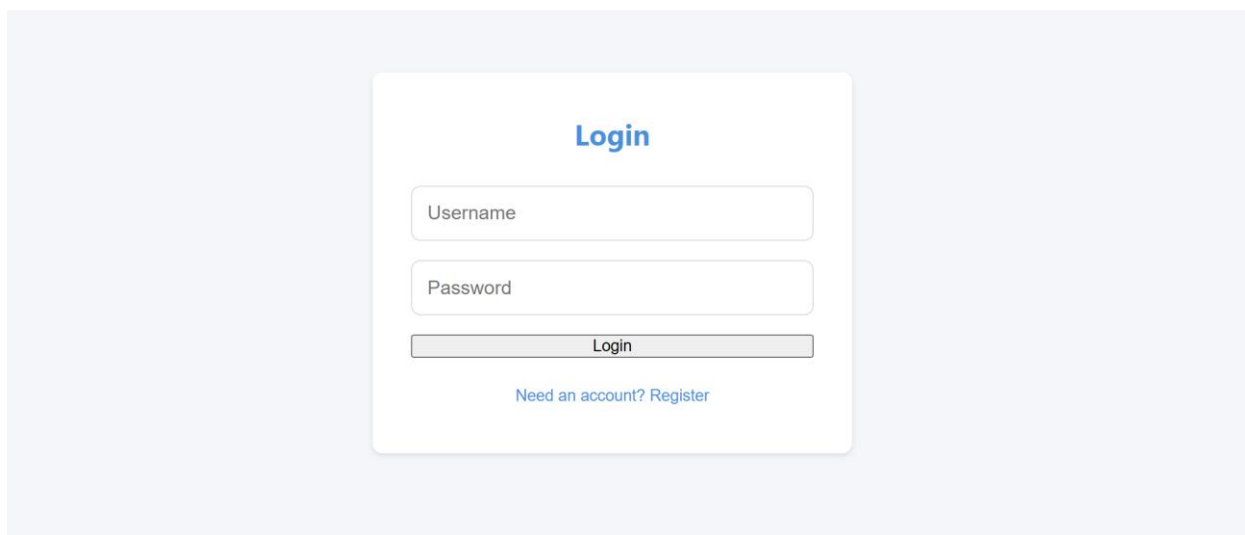


Рисунок 5.4 – Форма авторизации

5.1.6 Адаптивный дизайн

Интерфейс приложения адаптирован под различные размеры экранов. На мобильных устройствах элементы интерфейса перестраиваются в вертикальную структуру, а карточки задач занимают всю доступную ширину экрана. Формы создания и редактирования задач также адаптируются под размер экрана, сохраняя при этом удобство использования. Все интерактивные элементы имеют достаточный размер для комфортного взаимодействия на сенсорных экранах.

5.2 Управление программным средством

5.2.1 Создание задачи

Для создания новой задачи пользователю необходимо заполнить форму создания задачи, которая содержит обязательные и необязательные поля. В обязательные поля входит название задачи, которое должно быть заполнено для создания задачи. Дополнительно пользователь может указать описание задачи, дату выполнения, сложность задачи (Easy, Medium, Hard), теги через запятую и срок выполнения. После заполнения необходимых полей

пользователь нажимает кнопку "Add Task", после чего задача создается и автоматически отображается в общем списке задач.

5.2.2 Редактирование задачи

Редактирование существующей задачи осуществляется через кнопку "Edit", расположенную рядом с выбранной задачей. При нажатии на эту кнопку открывается форма редактирования, в которой пользователь может изменить все параметры задачи: название, описание, дату выполнения, сложность, теги и срок выполнения. После внесения необходимых изменений пользователь может сохранить их, нажав кнопку "Save", или отменить редактирование, нажав кнопку "Cancel". Все изменения автоматически синхронизируются с сервером.

5.2.3 Удаление задачи

Удаление задачи из системы выполняется с помощью кнопки "Delete", расположенной рядом с выбранной задачей. При нажатии на эту кнопку задача немедленно удаляется из списка и с сервера. Операция удаления является необратимой, поэтому пользователю следует быть внимательным при выполнении этого действия.

5.2.4 Изменение статуса задачи

Управление статусом задачи (выполнена/не выполнена) осуществляется через чекбокс, расположенный рядом с каждой задачей. При изменении статуса задачи система автоматически обновляет информацию на сервере. Выполненные задачи визуально отличаются от невыполненных: их заголовок перечеркивается, а прозрачность элемента снижается для лучшего визуального восприятия.

5.2.5 Фильтрация задач

Система предоставляет пользователю возможность фильтрации задач по различным параметрам. Пользователь может фильтровать задачи по уровню сложности (Easy, Medium, Hard), по тегам или по диапазону дат. Это позволяет эффективно организовать работу с большим количеством задач и быстро находить нужную информацию.

5.2.6 Аутентификация

Для работы с системой пользователю необходимо пройти процедуру авторизации. Вход в систему осуществляется через специальную форму авторизации, где пользователь вводит свои учетные данные. После успешной авторизации пользователь получает полный доступ к функционалу управления задачами. Для выхода из системы в верхней части интерфейса предусмотрена кнопка "Logout", при нажатии на которую происходит завершение сессии пользователя.

5.2.7 Просмотр задач

Задачи отображаются в виде структурированного списка, где каждая задача представлена в виде отдельного элемента. Для каждой задачи отображается полная информация: название, описание, дата создания, уровень сложности, теги, срок выполнения (если он установлен) и текущий статус выполнения. Такое представление информации позволяет пользователю быстро оценить состояние своих задач и принять необходимые меры по их выполнению.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы было разработано веб-приложение "Task Manager". Целью данной работы являлось создание современного и эффективного инструмента для управления задачами, который позволяет пользователям организовывать свою работу, отслеживать прогресс выполнения задач и эффективно взаимодействовать с другими участниками проекта, а также изучение современных технологий разработки веб-приложений, включая GraphQL, React и Node.js.

При разработке программного обеспечения были успешно выполнены все поставленные задачи. Система позволяет создавать, редактировать и удалять задачи с указанием заголовка и описания. Реализована возможность установки статуса выполнения задачи (выполнена/не выполнена) и назначения сложности задачи (легкая, средняя, сложная). Пользователи могут добавлять теги к задачам для категоризации и устанавливать сроки выполнения. Система предоставляет гибкие возможности фильтрации задач по сложности, тегам и диапазону дат. Реализована система регистрации и авторизации пользователей, а также привязка задач к конкретным пользователям и отслеживание даты создания задач.

Для успешного выполнения поставленных задач потребовалось изучить принципы работы GraphQL API, механизмы работы React и Node.js, а также современные подходы к разработке веб-приложений, включая управление состоянием, аутентификацию и безопасность. Особое внимание было уделено изучению архитектуры GraphQL и её преимуществ перед традиционными REST API, а также принципам работы с состоянием в React-приложениях.

Существует множество различных направлений развития данного веб-приложения. В будущем планируется добавить систему уведомлений о приближающихся сроках и реализовать совместную работу над задачами. Также рассматривается возможность добавления системы комментариев к задачам и внедрения системы приоритетов. Планируется реализовать возможность создания подзадач и внедрить систему меток и категорий. В перспективе возможно добавление возможности прикрепления файлов к задачам, внедрение системы отчетов и аналитики, реализация интеграции с другими сервисами и добавление возможности создания шаблонов задач.

Данное веб-приложение подходит для эффективного управления задачами как в небольших командах, так и в крупных организациях. Оно предоставляет удобный интерфейс для работы с задачами, гибкую систему фильтрации и поиска, а также надежную систему аутентификации и авторизации. Использование современных технологий, таких как GraphQL и React, обеспечивает высокую производительность и масштабируемость приложения.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- [1] GraphQL: The Definitive Guide: Understanding and Building APIs with GraphQL / Alex Banks, Eve Porcello – O'Reilly Media, 2023. – 456 p. ISBN 978-1-492-07432-4.
- [2] React: Up & Running: Building Web Applications / Stoyan Stefanov – O'Reilly Media, 2022. – 384 p. ISBN 978-1-492-07432-4.
- [3] TypeScript: Programming Language for JavaScript Applications / Boris Cherny – O'Reilly Media, 2023. – 512 p. ISBN 978-1-492-07432-4.
- [4] Node.js: The Complete Guide to Building RESTful APIs / Azat Mardan – Apress, 2023. – 428 p. ISBN 978-1-484-23958-3.
- [5] MongoDB: The Definitive Guide: Powerful and Scalable Data Storage / Shannon Bradshaw, Eoin Brazil, Kristina Chodorow – O'Reilly Media, 2023. – 560 p. ISBN 978-1-492-07432-4.
- [6] JWT Authentication: Secure User Authentication in Web Applications / Auth0 Documentation, 2023. [Electronic resource] – URL: <https://auth0.com/docs/secure/tokens/json-web-tokens>
- [7] Apollo Client: State Management for GraphQL / Apollo GraphQL Documentation, 2023. [Electronic resource] – URL: <https://www.apollographql.com/docs/react/>
- [8] Material-UI: React Component Library / MUI Documentation, 2023. [Electronic resource] – URL: <https://mui.com/material-ui/>
- [9] WebSocket: Real-time Communication Protocol / MDN Web Docs, 2023. [Electronic resource] – URL: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
- [10] Express.js: Web Application Framework for Node.js / Express.js Documentation, 2023. [Electronic resource] – URL: <https://expressjs.com/>

ПРИЛОЖЕНИЕ А

Текст программы

Исходный код проекта

Серверная часть

server/src/index.js

```
```javascript
```

```
const express = require('express');
const { ApolloServer } = require('apollo-server-express');
const cors = require('cors');
const typeDefs = require('./schema/typeDefs');
const resolvers = require('./resolvers');
const { getUserId } = require('./utils/auth');
```

```
async function startServer() {
```

```
 const app = express();
 app.use(cors());
```

```
 const server = new ApolloServer({
 typeDefs,
 resolvers,
 context: ({ req }) => ({
 req
 })
 });
```

```
 await server.start();
```

```

server.applyMiddleware({ app });

const PORT = process.env.PORT || 4000;

app.listen(PORT, () => {
 console.log(`Server running at
http://localhost:${PORT}${server.graphqlPath}`);
});
}

startServer();
...

server/src/schema/typeDefs.js
```javascript
const { gql } = require('apollo-server-express');

const typeDefs = gql`
  type User {
    id: ID!
    username: String!
    createdAt: String!
  }

  enum TaskComplexity {
    EASY
    MEDIUM
    HARD
  }

```

```
type Task {  
  id: ID!  
  title: String!  
  description: String  
  completed: Boolean!  
  createdAt: String!  
  userId: ID!  
  date: String!  
  complexity: TaskComplexity!  
  tags: [String!]!  
  deadline: String  
}
```

```
type AuthPayload {  
  token: String!  
  user: User!  
}
```

```
type Query {  
  tasks: [Task!]!  
  task(id: ID!): Task  
  me: User  
  tasksByComplexity(complexity: TaskComplexity!): [Task!]!  
  tasksByTag(tag: String!): [Task!]!  
  tasksByDateRange(startDate: String!, endDate: String!): [Task!]!  
}
```

```
type Mutation {  
  createTask(  

```



```

        title: String!
        description: String
        userId: String!
        date: String!
        complexity: TaskComplexity!
        tags: [String!]!
        deadline: String
    ): Task!
    updateTask(
        id: ID!
        title: String
        description: String
        completed: Boolean
        date: String
        complexity: TaskComplexity
        tags: [String!]
        deadline: String
    ): Task!
    deleteTask(id: ID!): Task!
    register(username: String!, password: String!): AuthPayload!
    login(username: String!, password: String!): AuthPayload!
}
`;

module.exports = typeDefs;
...

### server/src/resolvers/index.js
```javascript

```

```

const fs = require('fs');

const path = require('path');

const { generateToken, hashPassword, comparePassword, getUserId } =
require('../utils/auth');

const dataPath = path.join(__dirname, '../data');
const tasksPath = path.join(dataPath, 'tasks.json');
const usersPath = path.join(dataPath, 'users.json');

const readData = (filePath) => {
 const data = fs.readFileSync(filePath, 'utf8');
 return JSON.parse(data);
};

const writeData = (filePath, data) => {
 fs.writeFileSync(filePath, JSON.stringify(data, null, 2));
};

const resolvers = {
 Query: {
 tasks: (_, __, context) => {
 const userId = getUserId(context);
 const data = readData(tasksPath);
 return data.tasks.filter(task => task.userId === userId);
 },
 task: (_, { id }, context) => {
 const userId = getUserId(context);
 const data = readData(tasksPath);
 const task = data.tasks.find(task => task.id === id);
 }
 }
};

```

```

 if (!task || task.userId !== userId) {
 throw new Error('Task not found');
 }
 return task;
 },
 tasksByComplexity: (_, { complexity }, context) => {
 const userId = getUserId(context);
 const data = readData(tasksPath);
 return data.tasks.filter(task => task.userId === userId &&
task.complexity === complexity);
 },
 tasksByTag: (_, { tag }, context) => {
 const userId = getUserId(context);
 const data = readData(tasksPath);
 return data.tasks.filter(task => task.userId === userId &&
task.tags.includes(tag));
 },
 tasksByDateRange: (_, { startDate, endDate }, context) => {
 const userId = getUserId(context);
 const data = readData(tasksPath);
 return data.tasks.filter(task =>
 task.userId === userId &&
 task.date >= startDate &&
 task.date <= endDate
);
 },
 me: (_, __, context) => {
 const userId = getUserId(context);
 const data = readData(usersPath);

```

```

 const user = data.users.find(user => user.id === userId);

 if (!user) {
 throw new Error('User not found');
 }

 return {
 id: user.id,
 username: user.username,
 createdAt: user.createdAt
 };
 },
 Mutation: {
 createTask: async (_, { title, description, userId, date, complexity,
tags, deadline }, context) => {
 const contextUserId = getUserId(context);

 if (contextUserId !== userId) {
 throw new Error('Not authorized');
 }

 const data = readData(tasksPath);

 const newTask = {
 id: Date.now().toString(),
 title,
 description,
 completed: false,
 createdAt: new Date().toISOString(),
 userId,
 date,
 complexity,
 tags,

```

```

 deadline

 };

 data.tasks.push(newTask);

 writeData(tasksPath, data);

 return newTask;
 },

 updateTask: async (_, { id, title, description, completed, date,
 complexity, tags, deadline }, context) => {

 const userId = getUserId(context);

 const data = readData(tasksPath);

 const taskIndex = data.tasks.findIndex(task => task.id === id &&
 task.userId === userId);

 if (taskIndex === -1) {

 throw new Error('Task not found');

 }

 const updatedTask = {

 ...data.tasks[taskIndex],

 ...(title && { title }),

 ...(description && { description }),

 ...(completed !== undefined && { completed }),

 ...(date && { date }),

 ...(complexity && { complexity }),

 ...(tags && { tags }),

 ...(deadline && { deadline })

 };

 data.tasks[taskIndex] = updatedTask;

 writeData(tasksPath, data);

 return updatedTask;
 },

```

```

deleteTask: async (_, { id }, context) => {
 const userId = getUserId(context);
 const data = readData(tasksPath);
 const taskIndex = data.tasks.findIndex(task => task.id === id &&
task.userId === userId);
 if (taskIndex === -1) {
 throw new Error('Task not found');
 }
 const deletedTask = data.tasks[taskIndex];
 data.tasks.splice(taskIndex, 1);
 writeData(tasksPath, data);
 return deletedTask;
},
register: async (_, { username, password }) => {
 const data = readData(usersPath);
 if (data.users.find(user => user.username === username)) {
 throw new Error('Username already exists');
 }
 const hashedPassword = await hashPassword(password);
 const newUser = {
 id: Date.now().toString(),
 username,
 password: hashedPassword,
 createdAt: new Date().toISOString()
 };
 data.users.push(newUser);
 writeData(usersPath, data);
 const token = generateToken(newUser);
 return {

```

```

 token,
 user: {
 id: newUser.id,
 username: newUser.username,
 createdAt: newUser.createdAt
 }
 };
},
login: async (_, { username, password }) => {
 const data = readData(usersPath);
 const user = data.users.find(user => user.username === username);
 if (!user) {
 throw new Error('User not found');
 }
 const valid = await comparePassword(password, user.password);
 if (!valid) {
 throw new Error('Invalid password');
 }
 const token = generateToken(user);
 return {
 token,
 user: {
 id: user.id,
 username: user.username,
 createdAt: user.createdAt
 }
 };
}
}

```

```

};

module.exports = resolvers;
```

### server/src/utils/auth.js
```javascript
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');

const JWT_SECRET = 'your-secret-key'; // В реальном приложении должно быть
в .env

const generateToken = (user) => {
 return jwt.sign(
 { userId: user.id },
 JWT_SECRET,
 { expiresIn: '24h' }
);
};

const hashPassword = async (password) => {
 return await bcrypt.hash(password, 10);
};

const comparePassword = async (password, hashedPassword) => {
 return await bcrypt.compare(password, hashedPassword);
};

```



```

const getUserId = (context) => {
 const Authorization = context.req.headers.authorization;
 if (Authorization) {
 const token = Authorization.replace('Bearer ', '');
 const { userId } = jwt.verify(token, JWT_SECRET);
 return userId;
 }
 throw new Error('Not authenticated');
};

```

```

module.exports = {
 generateToken,
 hashPassword,
 comparePassword,
 getUserId
};
...

```

## Клиентская часть

### client/src/App.js

```

```javascript
import React, { useState, useEffect } from 'react';
import { ApolloClient, InMemoryCache, ApolloProvider, createHttpLink } from
 '@apollo/client';
import { setContext } from '@apollo/client/link/context';
import TaskList from './components/TaskList';
import CreateTask from './components/CreateTask';
import Auth from './components/Auth';

```

```

import './App.css';

const httpLink = createHttpLink({
  uri: 'http://localhost:4000/graphql'
});

const authLink = setContext((_, { headers }) => {
  const token = localStorage.getItem('token');
  return {
    headers: {
      ...headers,
      authorization: token ? `Bearer ${token}` : ''
    }
  };
});

const client = new ApolloClient({
  link: authLink.concat(httpLink),
  cache: new InMemoryCache()
});

function App() {
  const [user, setUser] = useState(null);

  useEffect(() => {
    const token = localStorage.getItem('token');
    const userId = localStorage.getItem('userId');
    if (token && userId) {
      setUser({ id: userId });
    }
  });
}

```

```

    }
  }, []));

const handleLogin = (userData, token) => {
  localStorage.setItem('token', token);
  localStorage.setItem('userId', userData.id);
  setUser(userData);
};

const handleLogout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('userId');
  setUser(null);
};

if (!user) {
  return (
    <ApolloProvider client={client}>
      <div className="app">
        <Auth onLogin={handleLogin} />
      </div>
    </ApolloProvider>
  );
}

return (
  <ApolloProvider client={client}>
    <div className="app">
      <header className="header">

```

```

    <div className="user-info">
      <h1>Task Manager</h1>
    </div>

    <button className="logout-button" onClick={handleLogout}>
      Logout
    </button>

  </header>

  <CreateTask />

  <TaskList />

</div>

</ApolloProvider>

);
}

```

```
export default App;
```

```
...
```

```
### client/src/components/Auth.js
```

```
```javascript
```

```
import React, { useState } from 'react';
```

```
import { useMutation } from '@apollo/client';
```

```
import { gql } from '@apollo/client';
```

```
const LOGIN = gql`
```

```
 mutation Login($username: String!, $password: String!) {
```

```
 login(username: $username, password: $password) {
```

```
 token
```

```
 user {
```

```
 id
```

```

 username
 }
 }
 }
};

```

```

const REGISTER = gql`
 mutation Register($username: String!, $password: String!) {
 register(username: $username, password: $password) {
 token
 user {
 id
 username
 }
 }
 }
`;

```

```

function Auth({ onLogin }) {
 const [isLoggedIn, setIsLoggedIn] = useState(true);
 const [username, setUsername] = useState('');
 const [password, setPassword] = useState('');
 const [error, setError] = useState('');

 const [login] = useMutation(LOGIN, {
 onCompleted: (data) => {
 onLogin(data.login.user, data.login.token);
 },
 onError: (error) => {

```

```

 setError(error.message);
 }
});

const [register] = useMutation(REGISTER, {
 onCompleted: (data) => {
 onLogin(data.register.user, data.register.token);
 },
 onError: (error) => {
 setError(error.message);
 }
});

const handleSubmit = async (e) => {
 e.preventDefault();
 setError('');

 try {
 if (isLogin) {
 await login({ variables: { username, password } });
 } else {
 await register({ variables: { username, password } });
 }
 } catch (err) {
 console.error('Error:', err);
 }
};

return (

```

```

<div className="auth-container">

 <h2>{isLogin ? 'Login' : 'Register'}</h2>

 {error && <p className="error">{error}</p>}

 <form onSubmit={handleSubmit} className="auth-form">

 <input
 type="text"
 value={username}
 onChange={(e) => setUsername(e.target.value)}
 placeholder="Username"
 required
 />

 <input
 type="password"
 value={password}
 onChange={(e) => setPassword(e.target.value)}
 placeholder="Password"
 required
 />

 <button type="submit">{isLogin ? 'Login' : 'Register'}</button>

 </form>

 <button
 className="toggle-auth"
 onClick={() => {
 setIsLogin(!isLogin);
 setError('');
 }}
 >

 {isLogin ? 'Need an account? Register' : 'Already have an account?
Login'}

```

```

 </button>

 </div>

);
}

export default Auth;
...

client/src/components/TaskList.js
```javascript
import React, { useState } from 'react';
import { useQuery, useMutation } from '@apollo/client';
import { GET_TASKS } from '../graphql/queries';
import { UPDATE_TASK, DELETE_TASK } from '../graphql/mutations';

function TaskList() {
    const [editingTask, setEditingTask] = useState(null);
    const [editTitle, setEditTitle] = useState('');
    const [editDescription, setEditDescription] = useState('');
    const [editDate, setEditDate] = useState('');
    const [editComplexity, setEditComplexity] = useState('MEDIUM');
    const [editTags, setEditTags] = useState('');
    const [editDeadline, setEditDeadline] = useState('');

    const { loading, error, data } = useQuery(GET_TASKS, {
        fetchPolicy: 'network-only'
    });

    const [updateTask] = useMutation(UPDATE_TASK, {

```



```

    refetchQueries: [{ query: GET_TASKS }]
  });

const [deleteTask] = useMutation(DELETE_TASK, {
  refetchQueries: [{ query: GET_TASKS }]
});

if (loading) return <p>Loading...</p>;
if (error) return <p>Error: {error.message}</p>;

const tasks = data.tasks;

const handleUpdateTask = async (taskId) => {
  try {
    const tagsArray = editTags.split(',').map(tag => tag.trim()).filter(tag
=> tag);

    await updateTask({
      variables: {
        id: taskId,
        title: editTitle,
        description: editDescription,
        date: editDate,
        complexity: editComplexity,
        tags: tagsArray,
        deadline: editDeadline || null
      }
    });

    setEditingTask(null);
  } catch (err) {

```

```

        console.error('Error updating task:', err);
    }
};

const handleDeleteTask = async (taskId) => {
    try {
        await deleteTask({
            variables: { id: taskId }
        });
    } catch (err) {
        console.error('Error deleting task:', err);
    }
};

const handleToggleComplete = async (task) => {
    try {
        await updateTask({
            variables: {
                id: task.id,
                completed: !task.completed
            }
        });
    } catch (err) {
        console.error('Error toggling task completion:', err);
    }
};

const startEditing = (task) => {
    setEditingTask(task.id);
};

```

```

    setEditTitle(task.title);
    setEditDescription(task.description || '');
    setEditDate(task.date);
    setEditComplexity(task.complexity);
    setEditTags(task.tags.join(', '));
    setEditDeadline(task.deadline || '');
  };

  if (tasks.length === 0) {
    return <div className="empty-message">No tasks yet. Create your first
task!</div>;
  }

  return (
    <div className="task-list">
      {tasks.map(task => (
        <div key={task.id} className={`task-item ${task.completed ?
'completed' : ''}`}>
          <input
            type="checkbox"
            checked={task.completed}
            onChange={() => handleToggleComplete(task)}
          />
          {editingTask === task.id ? (
            <div className="edit-form">
              <input
                type="text"
                value={editTitle}
                onChange={(e) => setEditTitle(e.target.value)}

```

```

        placeholder="Task title"
    />
<textarea
    value={editDescription}
    onChange={(e) => setEditDescription(e.target.value)}
    placeholder="Task description"
/>
<div className="form-row">
    <div className="form-group">
        <label>Date:</label>
        <input
            type="date"
            value={editDate}
            onChange={(e) => setEditDate(e.target.value)}
        />
    </div>
    <div className="form-group">
        <label>Complexity:</label>
        <select
            value={editComplexity}
            onChange={(e) => setEditComplexity(e.target.value)}
        >
            <option value="EASY">Easy</option>
            <option value="MEDIUM">Medium</option>
            <option value="HARD">Hard</option>
        </select>
    </div>
</div>
<div className="form-row">

```

```

    <div className="form-group">
      <label>Tags:</label>
      <input
        type="text"
        value={editTags}
        onChange={(e) => setEditTags(e.target.value)}
        placeholder="work, important, urgent"
      />
    </div>
    <div className="form-group">
      <label>Deadline:</label>
      <input
        type="date"
        value={editDeadline}
        onChange={(e) => setEditDeadline(e.target.value)}
      />
    </div>
  </div>
  <div className="edit-buttons">
    <button onClick={() =>
handleUpdateTask(task.id)}>Save</button>
    <button onClick={() => setEditingTask(null)}>Cancel</button>
  </div>
</div>
) : (
  <>
    <div className="task-content">
      <h3>{task.title}</h3>
      <p>{task.description}</p>

```

```

        <div className="task-meta">
            <span className="task-date">Date: {new
Date(task.date).toLocaleDateString()}</span>

            <span className="task-complexity">Complexity:
{task.complexity}</span>

            <div className="task-tags">
                {task.tags.map((tag, index) => (
                    <span key={index} className="tag">{tag}</span>
                ))}
            </div>

            {task.deadline && (
                <span className="task-deadline">
                    Deadline: {new
Date(task.deadline).toLocaleDateString()}

                    </span>
                )}

            <small>Created: {new
Date(task.createdAt).toLocaleString()}</small>

        </div>

    </div>

    <div className="task-actions">
        <button onClick={() => startEditing(task)}>Edit</button>

        <button onClick={() =>
handleDeleteTask(task.id)}>Delete</button>

    </div>

    </>

    )}

</div>

    )}

</div>

);

```

```

}

export default TaskList;
...

### client/src/components/CreateTask.js
```javascript
import React, { useState } from 'react';
import { useMutation } from '@apollo/client';
import { CREATE_TASK } from '../graphql/mutations';
import { GET_TASKS } from '../graphql/queries';

function CreateTask() {
 const [title, setTitle] = useState('');
 const [description, setDescription] = useState('');
 const [date, setDate] = useState(new Date().toISOString().split('T')[0]);
 const [complexity, setComplexity] = useState('MEDIUM');
 const [tags, setTags] = useState('');
 const [deadline, setDeadline] = useState('');

 const [createTask] = useMutation(CREATE_TASK, {
 refetchQueries: [{ query: GET_TASKS }],
 update(cache, { data: { createTask } }) {
 const { tasks } = cache.readQuery({ query: GET_TASKS });
 cache.writeQuery({
 query: GET_TASKS,
 data: { tasks: [...tasks, createTask] }
 });
 }
 });
}

```

```

});

const handleSubmit = async (e) => {
 e.preventDefault();
 if (!title.trim()) return;

 try {
 const userId = localStorage.getItem('userId');
 if (!userId) {
 console.error('User ID not found');
 return;
 }

 const tagsArray = tags.split(',').map(tag => tag.trim()).filter(tag =>
tag);

 await createTask({
 variables: {
 title,
 description: description || '',
 userId,
 date: date || new Date().toISOString().split('T')[0],
 complexity: complexity || 'MEDIUM',
 tags: tagsArray.length > 0 ? tagsArray : ['untagged'],
 deadline: deadline || null
 }
 });

 setTitle('');
 setDescription('');

```



```

 setDate(new Date().toISOString().split('T')[0]);
 setComplexity('MEDIUM');
 setTags('');
 setDeadline('');
 } catch (err) {
 console.error('Error creating task:', err);
 }
};

return (
 <form onSubmit={handleSubmit} className="create-task-form">
 <input
 type="text"
 value={title}
 onChange={(e) => setTitle(e.target.value)}
 placeholder="Task title"
 required
 />
 <textarea
 value={description}
 onChange={(e) => setDescription(e.target.value)}
 placeholder="Task description"
 />
 <div className="form-row">
 <div className="form-group">
 <label>Date:</label>
 <input
 type="date"
 value={date}

```

```

 onChange={(e) => setDate(e.target.value)}
 required
 />
</div>
<div className="form-group">
 <label>Complexity:</label>
 <select
 value={complexity}
 onChange={(e) => setComplexity(e.target.value)}
 required
 >
 <option value="EASY">Easy</option>
 <option value="MEDIUM">Medium</option>
 <option value="HARD">Hard</option>
 </select>
</div>
</div>
<div className="form-row">
 <div className="form-group">
 <label>Tags (comma-separated):</label>
 <input
 type="text"
 value={tags}
 onChange={(e) => setTags(e.target.value)}
 placeholder="work, important, urgent"
 />
 </div>
 <div className="form-group">
 <label>Deadline:</label>

```

```

 <input
 type="date"
 value={deadline}
 onChange={(e) => setDeadline(e.target.value)}
 />
 </div>
 </div>
 <button type="submit">Add Task</button>
 </form>
);
}

```

```
export default CreateTask;
```

```
...
```

```
client/src/graphql/queries.js
```

```
```javascript
```

```
import { gql } from '@apollo/client';
```

```
export const GET_TASKS = gql`
```

```
  query GetTasks {
```

```
    tasks {
```

```
      id
```

```
      title
```

```
      description
```

```
      completed
```

```
      createdAt
```

```
      userId
```

```
      date
```

```

        complexity
        tags
        deadline
    }
}
`;

```

```

export const GET_TASKS_BY_COMPLEXITY = gql`
    query GetTasksByComplexity($complexity: TaskComplexity!) {
        tasksByComplexity(complexity: $complexity) {
            id
            title
            description
            completed
            createdAt
            userId
            date
            complexity
            tags
            deadline
        }
    }
`;

```

```

export const GET_TASKS_BY_TAG = gql`
    query GetTasksByTag($tag: String!) {
        tasksByTag(tag: $tag) {
            id
            title

```

```

        description
        completed
        createdAt
        userId
        date
        complexity
        tags
        deadline
    }
}
`;

export const GET_TASKS_BY_DATE_RANGE = gql`
    query GetTasksByDateRange($startDate: String!, $endDate: String!) {
        tasksByDateRange(startDate: $startDate, endDate: $endDate) {
            id
            title
            description
            completed
            createdAt
            userId
            date
            complexity
            tags
            deadline
        }
    }
`;

```

```

export const GET_TASK = gql`
  query GetTask($id: ID!) {
    task(id: $id) {
      id
      title
      description
      completed
      createdAt
      userId
    }
  }
`;
...

```

```

### client/src/graphql/mutations.js
```javascript
import { gql } from '@apollo/client';

```

```

export const CREATE_TASK = gql`
 mutation CreateTask(
 $title: String!
 $description: String
 $userId: String!
 $date: String!
 $complexity: TaskComplexity!
 $tags: [String!]!
 $deadline: String
) {
 createTask(

```

```

 title: $title
 description: $description
 userId: $userId
 date: $date
 complexity: $complexity
 tags: $tags
 deadline: $deadline
) {
 id
 title
 description
 completed
 createdAt
 userId
 date
 complexity
 tags
 deadline
 }
}
`;

```

```

export const UPDATE_TASK = gql`
 mutation UpdateTask(
 $id: ID!
 $title: String
 $description: String
 $completed: Boolean
 $date: String
) {
 updateTask(
 id: $id
 title: $title
 description: $description
 completed: $completed
 date: $date
) {
 id
 title
 description
 completed
 createdAt
 userId
 date
 complexity
 tags
 deadline
 }
 }
`;

```

```

 $complexity: TaskComplexity
 $tags: [String!]
 $deadline: String
) {
 updateTask(
 id: $id
 title: $title
 description: $description
 completed: $completed
 date: $date
 complexity: $complexity
 tags: $tags
 deadline: $deadline
) {
 id
 title
 description
 completed
 createdAt
 userId
 date
 complexity
 tags
 deadline
 }
 }
}
`;

```

```

export const DELETE_TASK = gql`

```



```
mutation DeleteTask($id: ID!) {
 deleteTask(id: $id) {
 id
 }
}
`;
`;
```